THE ELEMENTS OF COMPUTING SYSTEMS

2의 보수법

정수의 덧셈을 컴퓨터로 구현하기

- 우리는 머릿속에서 손쉽게 정수의 덧셈을 계산한다.



2 + 3 = 5, (-3) + (-4) = -7, …

- 컴퓨터는 어떻게 하고 있을까?



010101110000…..?

정수의 덧셈을 컴퓨터로 구현하기

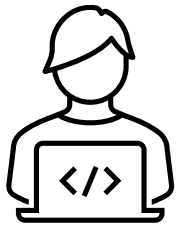
- 컴퓨터는 0과 1만을 사용해서 모든 것을 표현한다.
- 컴퓨터의 표현 단위는 비트. 1비트는 0 또는 1 둘 중 하나이다.
- 컴퓨터가 3비트 까지만 표현할 수 있다고 가정해 보자.



- 총 8가지 표현을 가진다.
- 000, 001, 010, 011, 100, 101, 110, 111

정수의 덧셈을 컴퓨터로 구현하기

- 우리가 원하는 상황은 다음과 같다.

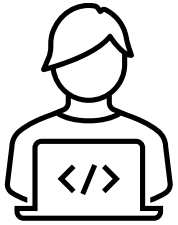


컴퓨터로 2 + 3 을 계산 해야지~ (컴퓨터에 2 + 3 입력)



2 + 3 이 들어왔군. 2와 3을 이진수로 변환하고
Overflow는 버리는 이진수 덧셈 연산을 수행한다.

결과가 나왔군. 이것을 십진수로 변환했더니 ??? 가 나왔어.



오 결과 ???은 내가 기대한 5와 일치하는군!

정수의 덧셈을 컴퓨터로 구현하기

- 우리가 명확히 해야 할 부분은 아래 밑줄 친 곳.



2 + 3 이 들어왔군. 2와 3을 **이진수로 변환하고** Overflow는 버리는 이진수 덧셈 연산을 수행한다.

결과가 나왔군. 이것을 **십진수로 변환했더니** ??? 가 나왔어.

- 정수를 이진수로 변환하는 함수를 알아야 함.
- 그 함수는 다음과 같은 성질을 만족해야 함. (함수는 f, Overflow버리는 이진수 연산은 *)
- 함수의 치역은 {000, 001, 010, 011, 100, 101, 110, 111}
- 함수의 정의역은 8개의 연속된 정수(띄엄띄엄 사용하면 불편하므로)
- 함수는 일대일 대응(역함수가 존재함)
- 제일 중요한 마지막 . f(a) * f(b) = f(a + b)

정수의 덧셈을 컴퓨터로 구현하기



2 + 3 이 들어왔군. 2와 3을 이진수로 변환하고 Overflow는 버리는 이진수 덧셈 연산을 수행한다.

결과가 나왔군. 이것을 십진수로 변환했더니 ??? 가 나왔어.

- 제일 중요한 마지막 . f(a) * f(b) = f(a + b)
- 마지막 식에서 양 변에 역함수를 적용하면 $f^{-1}(f(a) * f(b)) = a + b$
- 컴퓨터가 수행하는 과정을 그대로 나타낸 것이 $f^{-1}(f(2) * f(3)) = ???$
- 우리가 원하는 것은 ???이 2 + 3 = 5와 같기를 바라는 것이다.
- 최종적으로 아무 정수나 넣어도 그렇게 되길 바라므로 a, b를 사용해서 식을 나타낸 것.

- 하지만 사실 위 식을 만족하는 함수는 존재하지 않는다.

정수의 덧셈을 컴퓨터로 구현하기

- 제일 중요한 마지막 . f(a) * f(b) = f(a + b)
- 하지만 위 식을 만족하는 함수는 존재하지 않는다.
- 왜냐하면 a + b가 f의 정의역에 존재하지 않는 경우가 생기기 때문이다.
- 예를 들어 정의역이 {0, 1, 2, 3, 4, 5, 6, 7} 인 경우 3 + 5 = 8은 정의역에 없다.

- f(3) * f(5) = f(3 + 5), 정의되지 않음.
- f^{-1}(f(3) * f(5)) = 8, f의 역함수 f^{-1}의 range는 {0, 1, 2, 3, 4, 5, 6, 7} 임

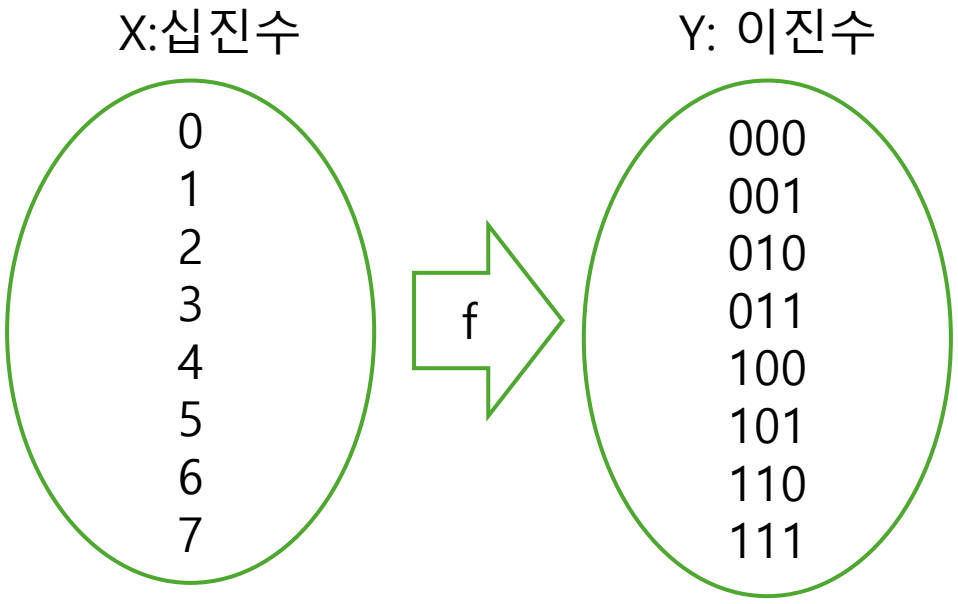


3 + 5 이 들어왔군. 3와 5을 이진수로 변환하고 Overflow는 버리는 이진수 덧셈 연산을 수행한다.

결과가 나왔군. 이것을 십진수로 변환했더니 … 8은 절대 나올 수 없다!

나올 수 있는 값은 0, 1, 2, 3, 4, 5, 6, 7 중에 하나다!

정수의 덧셈을 컴퓨터로 구현하기



- 제일 중요한 마지막 . f(a) * f(b) = f(a + b)

정의역의 임의의 원소 a, b에 대하여 위 식을 만족시키는 함수는 없음. a + b ∉ X 인 경우 f에서 정의되지 않기 때문

케이스는 2가지

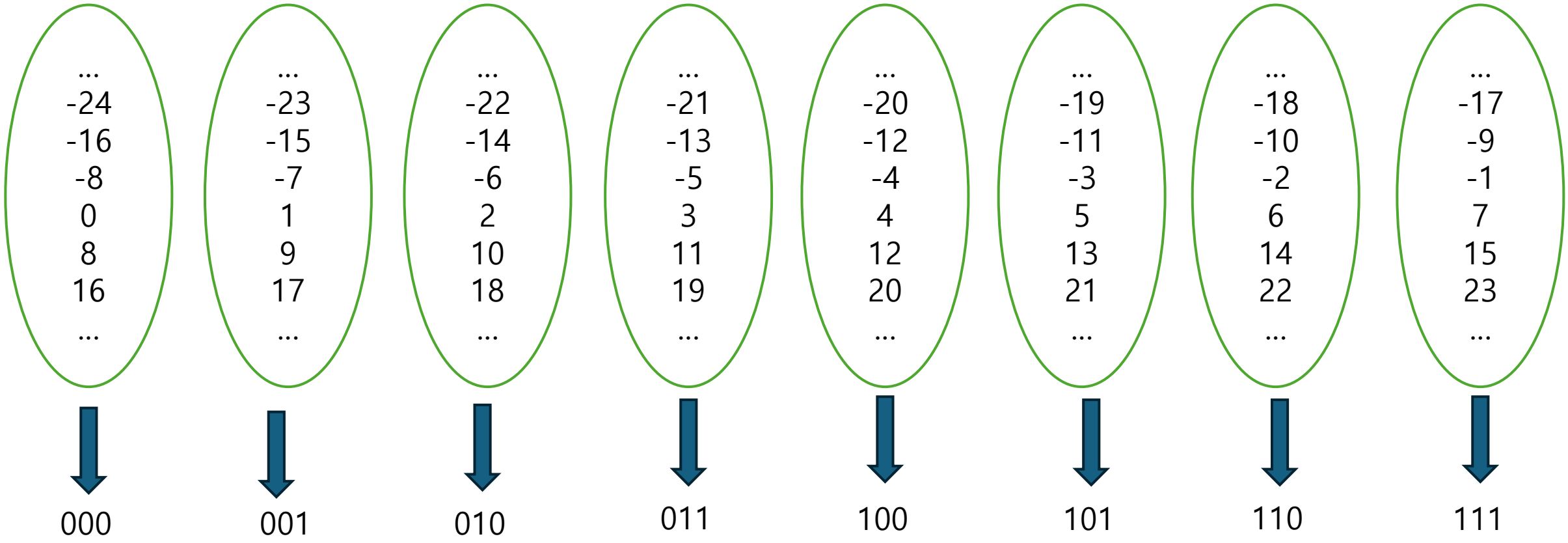
1. a + b ∈ X
2. a + b ∉ X

1번 경우만이라도 위 성질을 만족시킨다면 좋지 않을까?



그러면 좋을 듯? 근데 그런 함수가 뭐야?

정수의 덧셈을 컴퓨터로 구현하기



- 정수에서 3비트 이진수로 가는 함수를 다음과 같이 대응시키면 f(a) * f(b) = f(a + b)이 성립한다.
- 그러므로 이 함수 F의 정의역에서 연속된 정수 8개를 고르면 우리가 원하는 함수가 완성된다.

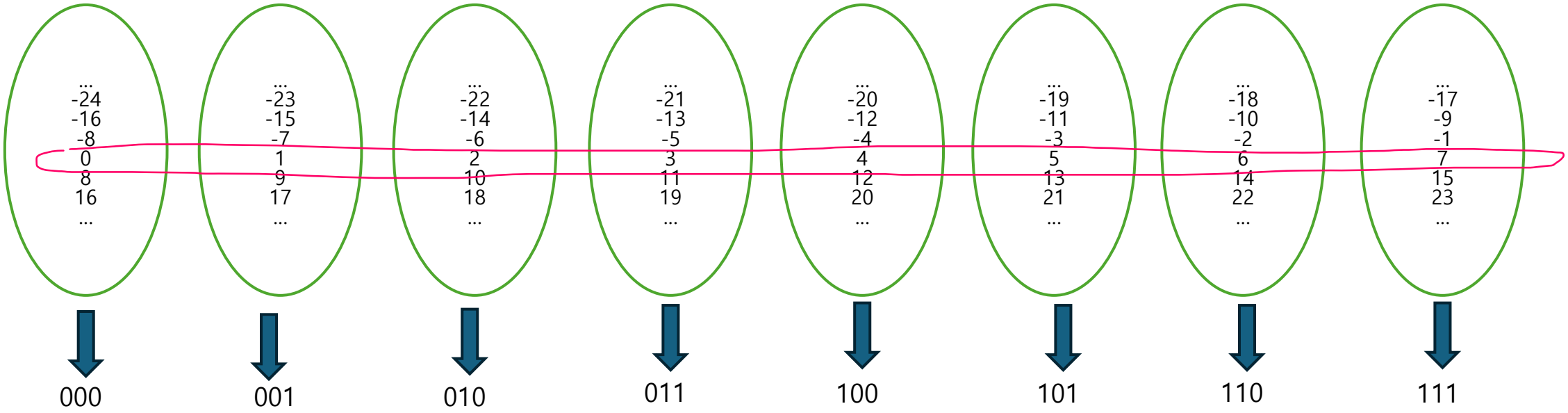
Overflow를 버리는 이진수 연산이란?

- 오른쪽 연산표가 바로 컴퓨터가 수행하는 Overflow를 버리는 이진수 덧셈 연산.
- 3bit만 사용하므로 4bit로 carry가 발생하면 버린다.
- 파란색으로 표시된 부분은 평소 생각하던 이진수 덧셈과는 다름.
- 일반적인 이진수 덧셈과 Overflow를 버리는 연산의 차이점.

+	000	001	010	011	100	101	110	111
000	000	001	010	011	100	101	110	111
001	001	010	011	100	101	110	111	000
010	010	011	100	101	110	111	000	001
011	011	100	101	110	111	000	001	010
100	100	101	110	111	000	001	010	011
101	101	110	111	000	001	010	011	100
110	110	111	000	001	010	011	100	101
111	111	000	001	010	011	100	101	110

정수의 덧셈을 컴퓨터로 구현하기

- 이진수 연산표를 최종적인 정수의 관점에서 살펴보자.
- 함수 F의 정의역에서 다음 처럼 7개의 연속적인 정수를 골랐다고 해보자.{0,1,2,3,4,5,6,7}



- f : {0, 1, 2, 3, 4, 5, 6, 7} => {000, 001, 010, 011, 100, 101, 110, 111}

정수의 덧셈을 컴퓨터로 구현하기

- 함수 f는 일대일 대응이므로 컴퓨터 사용자가 보는 정수의 연산표는 오른쪽과 같다.
- 000, 001, 010, 011, 100, 101, 110, 111



0 1 2 3 4 5 6 7

+	**000**	**001**	**010**	**011**	**100**	**101**	**110**	**111**
000	000	001	010	011	100	101	110	111
001	001	010	011	100	101	110	111	000
010	010	011	100	101	110	111	000	001
011	011	100	101	110	111	000	001	010
100	100	101	110	111	000	001	010	011
101	101	110	111	000	001	010	011	100
110	110	111	000	001	010	011	100	101
111	111	000	001	010	011	100	101	110



+	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	1	2	3	4	5	6	7	0
2	2	3	4	5	6	7	0	1
3	3	4	5	6	7	0	1	2
4	4	5	6	7	0	1	2	3
5	5	6	7	0	1	2	3	4
6	6	7	0	1	2	3	4	5
7	7	0	1	2	3	4	5	6

- 이진수의 연산표에서 각 이진수를 대응되는 정수로 바꿔서 쓴 것 뿐이다.

정수의 덧셈을 컴퓨터로 구현하기

- 함수 f는 $a + b \in X$ 인 원소에 대해서는 $f(a) * f(b) = f(a + b)$이 성립한다.
- 즉, 우리가 예상하는 정수의 덧셈처럼 작동한다.
- 오른쪽 표에서 표시한 부분과 같다.

+	**000**	**001**	**010**	**011**	**100**	**101**	**110**	**111**
000	000	001	010	011	100	101	110	111
001	001	010	011	100	101	110	111	000
010	010	011	100	101	110	111	000	001
011	011	100	101	110	111	000	001	010
100	100	101	110	111	000	001	010	011
101	101	110	111	000	001	010	011	100
110	110	111	000	001	010	011	100	101
111	111	000	001	010	011	100	101	110



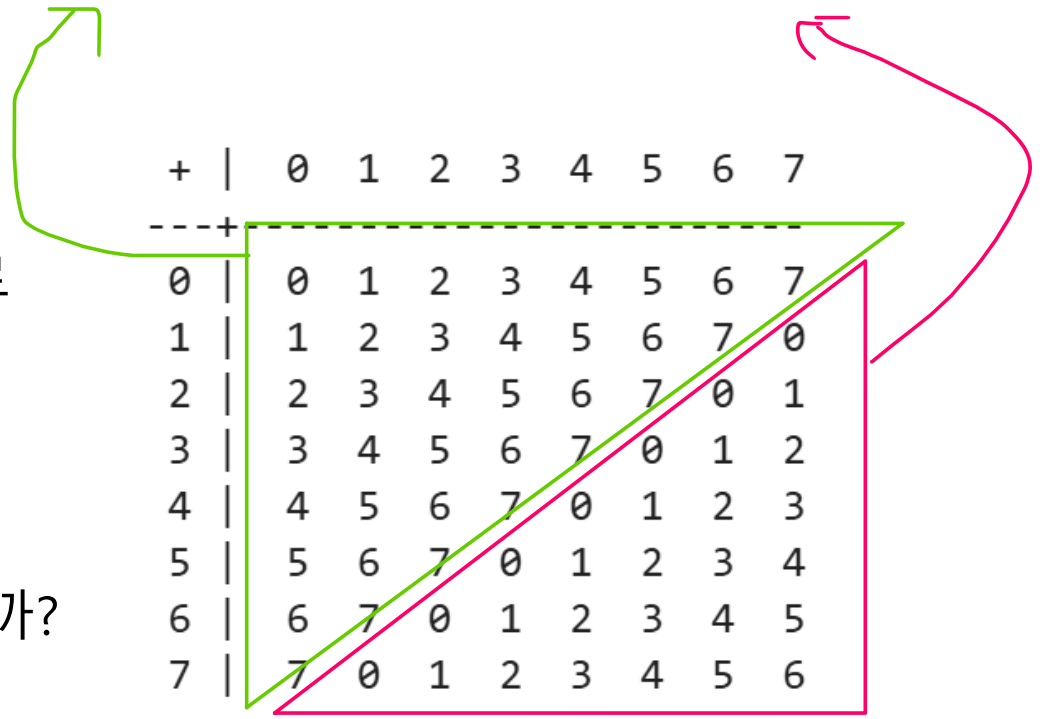
+	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	1	2	3	4	5	6	7	0
2	2	3	4	5	6	7	0	1
3	3	4	5	6	7	0	1	2
4	4	5	6	7	0	1	2	3
5	5	6	7	0	1	2	3	4
6	6	7	0	1	2	3	4	5
7	7	0	1	2	3	4	5	6

정수의 덧셈을 컴퓨터로 구현하기

- 정수에 대응시켰을때, 정수 덧셈 결과와 일치하는 부분도 있고 그렇지 않은 부분도 있음

- $a + b \in X$ 인 순서쌍 (a, b)에 대해서는
우리가 원하는대로 작동하지만 거의 절반의
순서쌍은 우리가 원하는대로 작동하지 않으므로
하드웨어를 낭비하는 느낌이 듬.

- $a + b \in X$ 인 순서쌍 (a, b)가 많아 지도록
함수 F의 정의역에서 7개를 더 잘 고를 수 있을까?



+	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	1	2	3	4	5	6	7	0
2	2	3	4	5	6	7	0	1
3	3	4	5	6	7	0	1	2
4	4	5	6	7	0	1	2	3
5	5	6	7	0	1	2	3	4
6	6	7	0	1	2	3	4	5
7	7	0	1	2	3	4	5	6

2의 보수법이란?

- 다른 여러가지 대응과 그에 따른 결과표.
- 예상과 다른 부분은 빨간색으로 표시함.

이진수		정수
000	<->	0
001	<->	1
010	<->	2
011	<->	3
100	<->	4
101	<->	5
110	<->	6
111	<->	-1

=== [2단계] 치환된 연산표 (음수 1개) ===

+	0	1	2	3	4	5	6	-1
0	0	1	2	3	4	5	6	-1
1	1	2	3	4	5	6	-1	0
2	2	3	4	5	6	-1	0	1
3	3	4	5	6	-1	0	1	2
4	4	5	6	-1	0	1	2	3
5	5	6	-1	0	1	2	3	4
6	6	-1	0	1	2	3	4	5
-1	-1	0	1	2	3	4	5	6

2의 보수법이란?

- 다른 여러가지 대응과 그에 따른 결과표.
- 예상과 다른 부분은 빨간색으로 표시함.

이진수		정수
000	<->	0
001	<->	1
010	<->	2
011	<->	3
100	<->	4
101	<->	5
110	<->	-2
111	<->	-1

=== [2단계] 치환된 연산표 (음수 2개) ===

+	0	1	2	3	4	5	-2	-1
0	0	1	2	3	4	5	-2	-1
1	1	2	3	4	5	-2	-1	0
2	2	3	4	5	-2	-1	0	1
3	3	4	5	-2	-1	0	1	2
4	4	5	-2	-1	0	1	2	3
5	5	-2	-1	0	1	2	3	4
-2	-2	-1	0	1	2	3	4	5
-1	-1	0	1	2	3	4	5	-2

2의 보수법이란?

- 다른 여러가지 대응과 그에 따른 결과표.
- 예상과 다른 부분은 빨간색으로 표시함.

이진수		정수
000	<->	0
001	<->	1
010	<->	2
011	<->	3
100	<->	4
101	<->	-3
110	<->	-2
111	<->	-1

=== [2단계] 치환된 연산표 (음수 3개) ===

+	0	1	2	3	4	-3	-2	-1
0	0	1	2	3	4	-3	-2	-1
1	1	2	3	4	-3	-2	-1	0
2	2	3	4	-3	-2	-1	0	1
3	3	4	-3	-2	-1	0	1	2
4	4	-3	-2	-1	0	1	2	3
-3	-3	-2	-1	0	1	2	3	4
-2	-2	-1	0	1	2	3	4	-3
-1	-1	0	1	2	3	4	-3	-2

2의 보수법이란?

- 다른 여러가지 대응과 그에 따른 결과표.
- 예상과 다른 부분은 빨간색으로 표시함.

이진수		정수
000	<->	0
001	<->	1
010	<->	2
011	<->	3
100	<->	-4
101	<->	-3
110	<->	-2
111	<->	-1

=== [2단계] 치환된 연산표 (음수 4개) ===

+	0	1	2	3	-4	-3	-2	-1
0	0	1	2	3	-4	-3	-2	-1
1	1	2	3	-4	-3	-2	-1	0
2	2	3	-4	-3	-2	-1	0	1
3	3	-4	-3	-2	-1	0	1	2
-4	-4	-3	-2	-1	0	1	2	3
-3	-3	-2	-1	0	1	2	3	-4
-2	-2	-1	0	1	2	3	-4	-3
-1	-1	0	1	2	3	-4	-3	-2

2의 보수법이란?

- 다른 여러가지 대응과 그에 따른 결과표.
- 예상과 다른 부분은 빨간색으로 표시함.

이진수		정수
000	<->	0
001	<->	1
010	<->	2
011	<->	-5
100	<->	-4
101	<->	-3
110	<->	-2
111	<->	-1

=== [2단계] 치환된 연산표 (음수 5개) ===

+	0	1	2	-5	-4	-3	-2	-1
0	0	1	2	-5	-4	-3	-2	-1
1	1	2	-5	-4	-3	-2	-1	0
2	2	-5	-4	-3	-2	-1	0	1
-5	-5	-4	-3	-2	-1	0	1	2
-4	-4	-3	-2	-1	0	1	2	-5
-3	-3	-2	-1	0	1	2	-5	-4
-2	-2	-1	0	1	2	-5	-4	-3
-1	-1	0	1	2	-5	-4	-3	-2

2의 보수법이란?

- 다른 여러가지 대응과 그에 따른 결과표.
- 예상과 다른 부분은 빨간색으로 표시함.

이진수		정수
000	<->	0
001	<->	1
010	<->	-6
011	<->	-5
100	<->	-4
101	<->	-3
110	<->	-2
111	<->	-1

=== [2단계] 치환된 연산표 (음수 6개) ===

+	0	1	-6	-5	-4	-3	-2	-1
0	0	1	-6	-5	-4	-3	-2	-1
1	1	-6	-5	-4	-3	-2	-1	0
-6	-6	-5	-4	-3	-2	-1	0	1
-5	-5	-4	-3	-2	-1	0	1	-6
-4	-4	-3	-2	-1	0	1	-6	-5
-3	-3	-2	-1	0	1	-6	-5	-4
-2	-2	-1	0	1	-6	-5	-4	-3
-1	-1	0	1	-6	-5	-4	-3	-2

2의 보수법이란?

- 다른 여러가지 대응과 그에 따른 결과표.
- 예상과 다른 부분은 빨간색으로 표시함.

이진수		정수
000	<->	0
001	<->	-7
010	<->	-6
011	<->	-5
100	<->	-4
101	<->	-3
110	<->	-2
111	<->	-1

=== [2단계] 치환된 연산표 (음수 7개) ===

+	0	-7	-6	-5	-4	-3	-2	-1
0	0	-7	-6	-5	-4	-3	-2	-1
-7	-7	-6	-5	-4	-3	-2	-1	0
-6	-6	-5	-4	-3	-2	-1	0	-7
-5	-5	-4	-3	-2	-1	0	-7	-6
-4	-4	-3	-2	-1	0	-7	-6	-5
-3	-3	-2	-1	0	-7	-6	-5	-4
-2	-2	-1	0	-7	-6	-5	-4	-3
-1	-1	0	-7	-6	-5	-4	-3	-2

2의 보수법이란?

- 확인 결과, 빨간색이 가장 적은 대응 2가지를 확인 할 수 있음.

```
=== [2단계] 치환된 연산표 (음수 3개) ===
 +  |   0   1   2   3   4  -3  -2  -1
----+--------------------------------
  0 |   0   1   2   3   4  -3  -2  -1
  1 |   1   2   3   4  -3  -2  -1   0
  2 |   2   3   4  -3  -2  -1   0   1
  3 |   3   4  -3  -2  -1   0   1   2
  4 |   4  -3  -2  -1   0   1   2   3
 -3 |  -3  -2  -1   0   1   2   3   4
 -2 |  -2  -1   0   1   2   3   4  -3
 -1 |  -1   0   1   2   3   4  -3  -2
```

```
=== [2단계] 치환된 연산표 (음수 4개) ===
 +  |   0   1   2   3  -4  -3  -2  -1
----+--------------------------------
  0 |   0   1   2   3  -4  -3  -2  -1
  1 |   1   2   3  -4  -3  -2  -1   0
  2 |   2   3  -4  -3  -2  -1   0   1
  3 |   3  -4  -3  -2  -1   0   1   2
 -4 |  -4  -3  -2  -1   0   1   2   3
 -3 |  -3  -2  -1   0   1   2   3  -4
 -2 |  -2  -1   0   1   2   3  -4  -3
 -1 |  -1   0   1   2   3  -4  -3  -2
```

- 둘중에서 실제로 사용되는것은 오른쪽 대응임.

2의 보수법이란?

- 왜 오른쪽을 사용할까? => 하드웨어 회로구성이 간편 해진다.
- 일단 앞에서 함수로 살펴보았듯이 정수의 덧셈을 완벽하게 구현할 수 없다.
- 왜냐하면 하드웨어는 유한개의 bit만 사용할 수 있고(우리의 예는 3bit) 정수는 무한하기 때문이다.
- 그렇기 때문에 최대한 정수의 덧셈과 유사하게 구현하면서도 빨간색이 나타나면 사용자에게 알려줘야한다. => 사용자가 기대하는 결과와 다른 것이기 때문에.

=== [2단계] 치환된 연산표 (음수 3개) ===

+	0	1	2	3	4	-3	-2	-1
0	0	1	2	3	4	-3	-2	-1
1	1	2	3	4	-3	-2	-1	0
2	2	3	4	-3	-2	-1	0	1
3	3	4	-3	-2	-1	0	1	2
4	4	-3	-2	-1	0	1	2	3
-3	-3	-2	-1	0	1	2	3	4
-2	-2	-1	0	1	2	3	4	-3
-1	-1	0	1	2	3	4	-3	-2

=== [2단계] 치환된 연산표 (음수 4개) ===

+	0	1	2	3	-4	-3	-2	-1
0	0	1	2	3	-4	-3	-2	-1
1	1	2	3	-4	-3	-2	-1	0
2	2	3	-4	-3	-2	-1	0	1
3	3	-4	-3	-2	-1	0	1	2
-4	-4	-3	-2	-1	0	1	2	3
-3	-3	-2	-1	0	1	2	3	-4
-2	-2	-1	0	1	2	3	-4	-3
-1	-1	0	1	2	3	-4	-3	-2

2의 보수법이란?

- 오른쪽의 경우 빨간색이 발생했을 때 쉽게 알아 볼 수 있는 규칙이 있음.
- 오른쪽 대응에서는 제일 왼쪽 비트가 0이면 양수, 1이면 음수이기 때문에 하드웨어 에서 빨간색이 발생했을 때 쉽게 검출 가능.
- 양수 + 양수 = 음수 또는 음수 + 음수 = 양수 2가지만 보면 되고 구별이 쉽다.
- Ex) 0 _ _ + 0 _ _ = 1 _ _ 이 나오면 빨간불. 1 _ _ + 1 _ _ = 0 _ _ 이 나오면 빨간불.

=== [2단계] 치환된 연산표 (음수 3개) ===

+	0	1	2	3	4	-3	-2	-1
0	0	1	2	3	4	-3	-2	-1
1	1	2	3	4	-3	-2	-1	0
2	2	3	4	-3	-2	-1	0	1
3	3	4	-3	-2	-1	0	1	2
4	4	-3	-2	-1	0	1	2	3
-3	-3	-2	-1	0	1	2	3	4
-2	-2	-1	0	1	2	3	4	-3
-1	-1	0	1	2	3	4	-3	-2

=== [2단계] 치환된 연산표 (음수 4개) ===

+	0	1	2	3	-4	-3	-2	-1
0	0	1	2	3	-4	-3	-2	-1
1	1	2	3	-4	-3	-2	-1	0
2	2	3	-4	-3	-2	-1	0	1
3	3	-4	-3	-2	-1	0	1	2
-4	-4	-3	-2	-1	0	1	2	3
-3	-3	-2	-1	0	1	2	3	-4
-2	-2	-1	0	1	2	3	-4	-3
-1	-1	0	1	2	3	-4	-3	-2